Supplemental Channel Partner Instructions: Compare

This document supplements the Channel Partner Instructions delivered with your compare page.

It covers three Buy button options, each of which can be set for the whole table or per product:

- Updating the buy button styling
- Setting customizations for every button
- Preserving click analytics

Updating the buy button styling

In order to update a buy button's styling, you will need to update the html as well as adding a `js` snippet.

The snippet replaces the compare component's built-in buy button renderer. Customizations can be set in two places:

- **Per product**, on the button item inside `channel-html-compare.json` — use this when products need different classes, attributes, or unavailable messaging.
- **For every product**, on the `data-compare-button-customization` attribute in `index.html` — use this when the whole table shares one style.

When both are set, the per-product values win.

Setting customizations for every button

1. Open `index.html`
2. Add your custom CSS class to the `<style>` tag
3. Update the `<div>` for the channel-compare to include the `data-compare-button-customization` attribute with your own custom classes and attributes ex:

```
<div
  class="channel-compare"
  data-compare-button-customization='{"button":{"classes":["[YOUR CLASS NAME]"],"attrs":
{"target":"_blank"}},"notAvailable":"<span class=\"[YOUR CLASS NAME]\">Not available</span>"}'
></div>
```

Available fields on the config object

Key	Type	Required	Description
button	object	No	Customizes the <a> anchor when a product has a buy link. Omit to leave the button untouched.
button.classes	array of strings	No	Replaces the anchor's default classes (button compare-button). Omit to keep the defaults. `compare-button` is always re-added for click tracking.
button.attrs	object of string:string	No	Attributes added on top of the anchor's built-in ones (aria-label, href). Overwrites any built-in attribute with the same name except `id`.
notAvailable	string	No	HTML rendered inside `.mdash-container` instead of the default em-dash ("Not Applicable") when a product has no buy link. Omit to keep the default fallback.

NOTE: `id` cannot be overridden. The compare component reports it as the product name in click analytics.

Setting customizations per product

1. Open the `channel-html-compare.json` file

2. Search for the Product you want to change, then its `"buy"` section
3. Add any of the fields below to the button item ex:

```
"buy": [
  {
    "template": "preview-item-button",
    "aria": "__i18n_X__",
    "value": "__i18n_XX__",
    "featureTitleOverride": "__i18n_XXXX__",
    "classes": ["[YOUR CLASS NAME]"],
    "attrs": { "target": "_blank" },
    "notAvailable": "<span class=\"[YOUR CLASS NAME]\">Not available</span>"
  }
]
```

Available fields on the button item

Key	Type	Required	Description
classes	array of strings	No	Replaces the anchor's default classes (button compare-button) for this product. Takes priority over `button.classes`. `compare-button` is always re-added for click tracking.
attrs	object of string:string	No	Attributes added on top of the anchor's built-in ones (aria-label, href). Takes priority over `button.attrs` for matching keys. `id` cannot be overridden.
notAvailable	string	No	HTML rendered inside `.mdash-container` when this product has no buy link. Takes priority over the top-level `notAvailable`.

NOTE: `notAvailable` is inserted as HTML, so tags in the string are rendered as markup. Only use values you author yourself.

Adding the snippet

4. Locate the data section marked by the comment boundaries:

```
<!-- BEGIN > CHANNEL SCRIPTS -->
...
<!-- END > CHANNEL SCRIPTS -->
```

5. Right before the `<!-- END > CHANNEL SCRIPTS -->` add the following JS snippet

Rather than copying out of this PDF, the same snippet ships alongside it in the `snippets` folder, in two forms:

File	Use
`update-compare-button-v2.1.2.html`	Readable. Identical to the code printed below.
`update-compare-button-v2.1.2.min.html`	Minified. Same behaviour, roughly half the size.

Either one can be pasted in as-is.

```
<script type="text/javascript" id="update-compare-button">
  (function () {
    "use strict";

    var TRACKING_CLASS = "compare-button";
    var MAX_ATTEMPTS = 100;
    var attempts = 0;

    function readButtonCustomization() {
      var el = document.querySelector(".channel-compare");
      var attr = el && el.getAttribute("data-compare-button-customization");
      if (!attr) return {};

      try {
        return JSON.parse(attr);
      } catch (e) {
        console.warn("[compare-button-customization] invalid JSON", e);
        return {};
      }
    }

    function withTrackingClass(classes) {
      var list = classes.slice();
      if (list.indexOf(TRACKING_CLASS) === -1) list.push(TRACKING_CLASS);
      return list;
    }

    function notifyDomUpdated() {
      window.dispatchEvent(new CustomEvent("channel:reorder-dom"));
    }

    function overrideButtonRenderer(instance) {
      if (typeof instance._renderCompareButton !== "function") return;

      var config = readButtonCustomization();
      var buttonConfig = config.button || {};

      instance._renderCompareButton = function ({ item, productName }) {
        var {
          aria,
          value,
          featureTitleOverride,
          classes,
          attrs,
          notAvailable,
        } = item;

        var helpers = (window.channelHtml && window.channelHtml.helpers) || {};
        var productId = helpers.stringToId
          ? helpers.stringToId(productName)
          : productName;
        var ariaLabel = aria || "Not Applicable";
        var notAvailableContent = notAvailable || config.notAvailable;

        return instance._createElement("div", {
          classes: ["button-container"],
          children: [
            value
              ? instance._createElement("a", {
                  classes: withTrackingClass(
                    classes ||
                      buttonConfig.classes || ["button", TRACKING_CLASS]
                  ),
                  attrs: {
                    "aria-label": aria,
                    href: value,
                    ...(buttonConfig.attrs || {}),
                    ...(attrs || {}),
                    id: productId,
                  },
                  content: featureTitleOverride || "",
                })
              : instance._createElement("div", {
                  classes: ["mdash-container"],
                  ...(notAvailableContent
                    ? { content: notAvailableContent }
                    : {
                        children: [
                          instance._createElement("p", {
```

```
                          classes: ["mdash", "copy"],
                          attrs: { "aria-hidden": "true" },
                          content: "&mdash;",
                        }),
                        instance._createElement("p", {
                          classes: ["visuallyhidden"],
                          content: ariaLabel,
                        }),
                      ],
                    }),
                }),
            ],
          });
        };
      }

      function rerenderAllColumns(instance) {
        if (!instance.dropdowns || !instance.dropdowns.length) {
          console.warn(
            "[compare-button-customization] no dropdowns; skipped re-render"
          );
          return;
        }

        instance.dropdowns.forEach(function (dropdown, columnIndex) {
          instance._updateProduct(columnIndex, Number(dropdown.value));
        });

        notifyDomUpdated();
      }

      function dropdownsReady(instance) {
        if (!instance.dropdowns || !instance.dropdowns.length) return true;
        return Array.prototype.every.call(instance.dropdowns, function (d) {
          return d.options.length > 0 && !Number.isNaN(Number(d.value));
        });
      }

      function waitForCompareInstance() {
        var registry =
          window.channelHtml &&
          window.channelHtml.channelCompare &&
          window.channelHtml.channelCompare.instances;

        var id = registry && Object.keys(registry)[0];
        var instance = id && registry[id];

        if (!instance || !instance.compareData || !dropdownsReady(instance)) {
          if (++attempts > MAX_ATTEMPTS) return;
          setTimeout(waitForCompareInstance, 100);
          return;
        }

        overrideButtonRenderer(instance);
        rerenderAllColumns(instance);
      }

      window.addEventListener("pageshow", function () {
        notifyDomUpdated();
      });

      document.addEventListener("change", function (event) {
        var target = event.target;
        if (!target || !target.classList) return;
        if (!target.classList.contains("selector-dropdown")) return;
        requestAnimationFrame(notifyDomUpdated);
      });

      if (document.readyState === "loading") {
        document.addEventListener("DOMContentLoaded", waitForCompareInstance);
      } else {
        waitForCompareInstance();
      }
    })();
  </script>
```

Preserving click analytics

The compare component tracks Buy button clicks by attaching listeners to elements matching `.compare-button`. Two things in this flow can silently break that tracking, and the snippet above handles both:

- **The `compare-button` class is always kept on the anchor.** Even if `classes` omits it, the snippet appends it. This class carries no styling of its own — it exists only as the analytics hook — so keeping it does not affect your design. Restyle freely by replacing `button` with your own classes.
- **The component is told to re-bind after every re-render.** Applying your customizations replaces the Buy button elements, which discards the listeners that were attached to the originals. The snippet dispatches `channel:reorder-dom` after each re-render so the listeners are re-attached to the new elements.

Do not remove the `withTrackingClass`, `notifyDomUpdated`, `pageshow`, or `change` logic from the snippet — each one keeps tracking alive on a different path (initial load, product swap, and back/forward navigation).

Verifying tracking still works

After adding the snippet, load the page and run this in the browser console:

```
window.addEventListener("channel:buy-button-click", (e) =>
  console.log("tracked:", e.detail)
);
```

Click each Buy button — every click should log a `tracked:` line with the product's `productName`. Then change a product in a dropdown and click the Buy buttons again; all of them should still log. If any click produces no log, tracking is broken for that button.